Week 2 — Level 2: Intermediate

Functions That Give Back

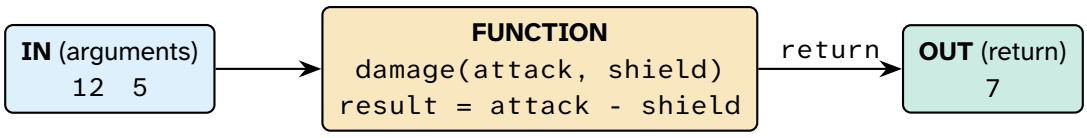
return hands an answer back; parameters make one function work for many jobs.

What you will learn

- How **parameters** let one function do many different jobs.
- How `return` hands an answer back, so your program can use it.
- Why printing an answer is not the same as returning it.
- How to write a **docstring** that explains your function.

The big idea: a function is a machine

A function is like a machine. Values go **in** through the parameters. The machine works on them. An answer comes **back out** with `return`.



A function with two parameters that returns an answer

```
def damage(attack, shield):
    """Return the damage a hit does."""
    return attack - shield

hit = damage(12, 5)       # 12 goes into attack, 5 goes into shield
print(f"Ouch! {hit} damage")
```

The values in the brackets when you *call* the function (`12` and `5`) are **arguments**. They fill the **parameters** (`attack` and `shield`) in the same order.

Print or return?

`print(answer)`	`return answer`
Shows the answer on the screen.	Hands the answer back to the code that called the function.
Like shouting it out of a window: you cannot keep it.	Like putting it in your hand: you can store it or do math with it.
The function gives back `None` (nothing).	The function gives back the answer.

Bug hunt clue: if you see the word `None` where a number should be, a function probably *printed* instead of *returning*.

Docstrings: notes for future-you

A **docstring** is a sentence in triple quotes just under the `def` line. It says what the function gives back. Thonny shows it when you hover over the function name, and your friends can read it to understand your code.

What happens in a function stays in a function

Variables you make inside a function (like `result`) only exist inside it. Outside the function, Python says `NameError`. If you need the value outside, `return` it.

Exercise 2.1: Game Math

Folder: `exercises/week02/ex-game-math`. Open `starter.py` and press **Run**.

1. Find the word `None` in the output. Which function caused it? _____
2. TODO 1: change `print(...)` to `return ...` in `xp_needed()`.
3. Predict first, then finish `damage()` (TODO 2 and 3): `damage(12, 5)` = _____ `damage(3, 5)` = _____
4. TODO 4: finish `is_critical()`. Tip: `return roll >= 19` gives back True or False by itself.
5. TODO 5: remove the # from the two health lines. Health left: _____
6. Check your work with `check_my_work.py`.

Exercise 2.2: Flower Garden

Folder: `exercises/week02/ex-flower-garden`. Press **Run**: all three flowers land in the same spot with 4 petals.

1. TODO 1: `petal_turn(petals)` should return `360 / petals`. How far does it turn for 6 petals? _____
2. TODO 2 and 3: use `t.goto(x, y)` so each flower appears where it was asked to.
3. TODO 4: repeat once for each petal: `range(petals)`.
4. Fill in the arguments you used for your favorite flower:

   `draw_flower(t, x =` _____`, y =` _____`, petals =` _____`, color =` _____`)`

Vocabulary box

parameter A named slot in a function's `def` line, like `attack`. It receives a value.

argument The value you hand in when you call the function, like `12`.

return Hands an answer back to the code that called the function, and ends the function.

None Python's word for "nothing". A function with no `return` gives back `None`.

docstring A sentence in triple quotes under `def` that explains what the function does.

scope Where a variable can be seen. Variables made inside a function stay inside it.

Reflection

Explain print versus return to someone who missed today (use your own example):

Write the docstring you are most proud of:

Week 2 Level 2: Intermediate

Functions That Give Back

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): What comes back? (paper, 3 minutes)

Write what each call gives back. Write `None` if it gives back nothing.

```
def add_three(n):
    return n + 3

def shout(word):
    print(word + "!")
```

`add_three(4)` gives back _____

`add_three(add_three(1))` gives back _____

`shout("hi")` gives back _____

★ Core (everyone): Potion Shop Calculator (Thonny, 10 minutes)

Open `exercises/week02/hw-shop-calculator/starter.py`. You have 50 coins and potions cost 12 coins each.

1. TODO 1: `total_cost()` returns price times quantity.
2. TODO 2: `can_afford()` returns True or False.
3. TODO 3: write a docstring for `coins_left()`.
4. TODO 4: finish `coins_left()`. You can call `can_afford()` inside it!
5. Run it. Buying 3 potions leaves _____ coins. Buying 5 potions leaves _____ coins.
6. Open `check_my_work.py` and press **Run**.

★★ Challenge (want more?): Sale day

Add a function `discount(cost)` that returns the cost minus 5 if the cost is 40 or more, and the same cost otherwise. Use it in the main program before checking `can_afford()`. Write a docstring for it. What do 5 potions cost on sale day? _____

Creative (make it yours): Invent a function machine

Invent a secret rule for a function machine (for example “add the two numbers, then double”). Try it on someone at home: they hand you values, you hand back answers, they guess the rule. Then write your rule as a Python function with a docstring:

Self-check: tick each box when it is true

- ☐ I can explain the difference between `print` and `return`.
- ☐ All three shop functions return their answer (no `None` in my output).
- ☐ Every function in my homework has a docstring.
- ☐ `check_my_work.py` shows `[PASS]` for my shop calculator.